Homework 1 - Simple Linear Regression

Libraries and setup

```
library(tidyverse)
library(tidymodels)
library(ggformula)
library(ISLR)
library(MASS)

set.seed(1998)
```

Question 1. Simple Linear Regression with the Auto dataset

We analyze the relationship between miles per gallon (`mpg`) and horsepower (`horsepower`) using a simple linear regression model.

a) Perform a simple linear regression with mpg as the response and horsepower as the predictor. Comment on the output.

i. Is there a relationship between the predictor and the response?

ii. How strong is the relationship between the predictor and the response?

iii. Is the relationship between the predictor and the response positive or negative?

iv. What is the predicted mpg associated with a horsepower of 98? What are the associated 95 % confdence and prediction intervals?

(b) Plot the response and the predictor. Use the abline() function to display the least squares regression line.

(c) Use the plot() function to produce diagnostic plots of the least squares regression ft. Comment on any problems you see with the ft.

(a) Fit the model

```
# incase of missing values, remove them
Auto_clean <- Auto |>
  drop_na(mpg, horsepower)

# Linear regression specification
lm_spec <- linear_reg() |>
  set_mode("regression") |>
  set_engine("lm")

# Fit the simple linear regression model
auto_fit <- lm_spec |>
  fit(mpg ~ horsepower, data = Auto_clean)
```

```r
tidy(auto_fit)
```

```
## # A tibble: 2 x 5
##   term        estimate std.error statistic   p.value
##   <chr>          <dbl>     <dbl>     <dbl>     <dbl>
## 1 (Intercept)   39.9     0.717        55.7 1.22e-187
## 2 horsepower    -0.158   0.00645     -24.5 7.03e- 81
```

The fitted model has the form

$$\widehat{mpg} = \beta_0 + \beta_1 \cdot horsepower$$
$$\widehat{mpg} = 39.94 - 0.16 \cdot horsepower$$

```r
glance(auto_fit)
```

```
## # A tibble: 1 x 12
##   r.squared adj.r.squared sigma statistic  p.value    df logLik   AIC   BIC
##       <dbl>         <dbl> <dbl>     <dbl>    <dbl> <dbl>  <dbl> <dbl> <dbl>
## 1     0.606         0.605  4.91      600. 7.03e-81     1 -1179. 2363. 2375.
## # i 3 more variables: deviance <dbl>, df.residual <int>, nobs <int>
```

Interpretation

- The p-value for `horsepower` is extremely small, so there is strong evidence of a relationship between horsepower and mpg.
- The estimated slope is negative, which means that as horsepower increases, mpg tends to decrease.
- The R-squared value shows that horsepower explains a meaningful portion of the variability in mpg, but not all of it, which is reasonable since fuel efficiency depends on more than just horsepower.

(b) Prediction at horsepower = 98

```r
new_obs <- tibble(horsepower = 98)

cbind(
predict(auto_fit, new_data = new_obs),# prediction for the mean response
predict(auto_fit, new_data = new_obs, type = "conf_int"),# 95% confidence interval for the mean respons
predict(auto_fit, new_data = new_obs, type = "pred_int"))# 95% prediction interval for an individual ca
```

```
##      .pred .pred_lower .pred_upper .pred_lower .pred_upper
## 1 24.46708    23.97308    24.96108     14.8094    34.12476
```

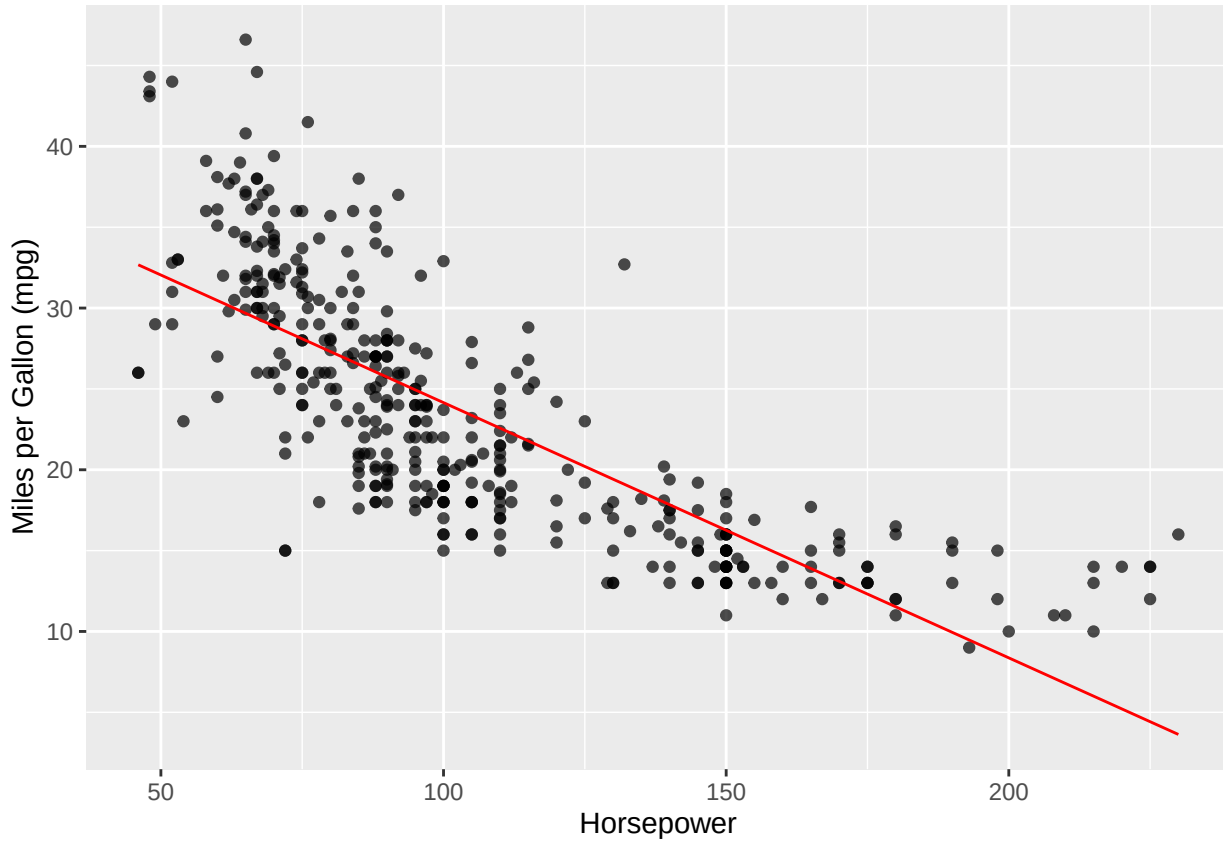
Interpretation

- The confidence interval gives a range for the *average* mpg of cars with horsepower 98.
- The prediction interval is wider because it reflects the uncertainty for a single individual car.

(c) Scatterplot with regression line

```r
auto_fit |>
  augment(new_data = Auto_clean) |>
  ggplot(aes(x = horsepower, y = mpg)) +
```

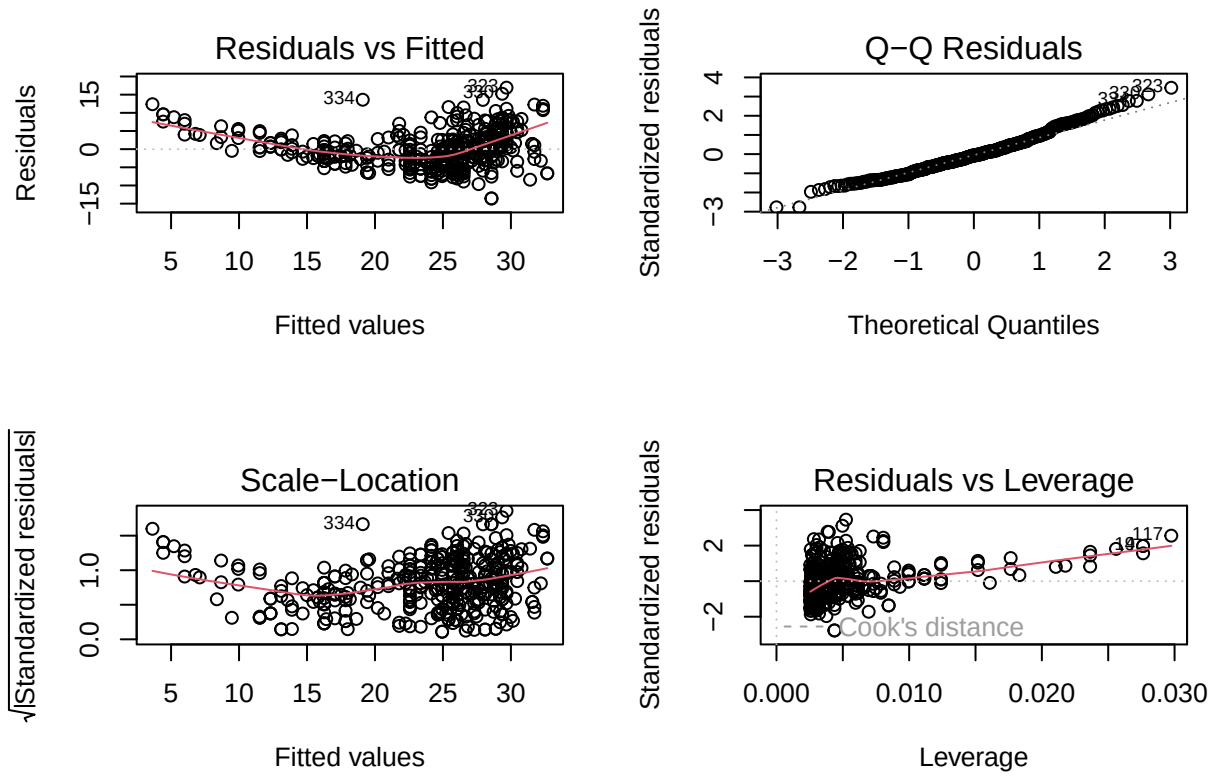
```r
  geom_point(alpha = 0.7) +
  geom_line(aes(y = .pred), color = "red") +
  labs(x = "Horsepower", y = "Miles per Gallon (mpg)")
```



The downward trend in the plot is consistent with the negative slope estimated by the model.

(d) Diagnostic plots

```r
par(mfrow = c(2, 2))
auto_fit |>
  extract_fit_engine() |>
  plot()
```




```r
par(mfrow = c(1, 1))
```

Comments

- The residuals vs fitted plot suggests some curvature, indicating the relationship may not be perfectly linear.
- The Q-Q plot shows mild departures from normality in the tails.
- There are a few points with higher leverage, but none appear extreme enough to dominate the fit.

Overall, the linear model captures the main trend, but it is clearly a simplification.

Question 2. Linear vs Cubic Regression (Conceptual)

We compare:

- a linear model: $Y = \beta_0 + \beta_1 X + \epsilon$
- a cubic model: $Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \epsilon$

(a) Which model is more flexible? Why?

The cubic model is more flexible because it can represent curved relationships, while the linear model is restricted to a straight line.

(b) Suppose that the true relationship between X and Y is linear, i.e. $Y = \beta_0 + \beta_1 X + \epsilon$.

(i) Based on model flexibility, which model would you expect to have a lower training RSS? Why?

The cubic model is expected to have a lower (or equal) training RSS because its extra flexibility allows it to fit the training data more closely.

(ii) Based on model flexibility, which model would you expect to have a lower test RSS? Why?

The linear model is expected to have a lower test RSS because it matches the true underlying relationship and avoids overfitting noise.

Question 3. Simple Linear Regression with the Boston dataset (TidyModels)

Include a full analysis similar to what we did for the Human Freedom Index data in class. Key components should include the following (but not limited to):

a) Fit a simple linear regression model with `medv` as your response and `lstat` as your predictor.

b) Perform a train/test split to assess predictive performance.

c) Report regression output (using `TidyModels`) and construct confidence intervals for model coefficients and interpret

d) Provide a short discussion of whether the predictor provides a reasonable model for `medv`. Include model diagnostics etc.

We model median home value (`medv`) as a function of the percentage of lower-status population (`lstat`), following the same workflow used in the class SLR notebook.

(a) Model specification and fit

```r
# Prepare data
boston_df <- Boston |>
  dplyr::select(medv, lstat)

# Linear regression specification
lm_spec <- linear_reg() |>
  set_engine("lm")

# Fit on full data
boston_fit <- lm_spec |>
  fit(medv ~ lstat, data = boston_df)
```

```r
# Model summary using broom helpers
tidy(boston_fit)
```

```
## # A tibble: 2 x 5
##   term        estimate std.error statistic   p.value
##   <chr>          <dbl>     <dbl>     <dbl>     <dbl>
## 1 (Intercept)   34.6      0.563       61.4 3.74e-236
## 2 lstat         -0.950    0.0387     -24.5 5.08e- 88
```

```r
glance(boston_fit)
```

```
## # A tibble: 1 x 12
##   r.squared adj.r.squared sigma statistic  p.value    df logLik   AIC   BIC
##       <dbl>         <dbl> <dbl>     <dbl>    <dbl> <dbl>  <dbl> <dbl> <dbl>
## 1     0.544         0.543  6.22      602. 5.08e-88     1 -1641. 3289. 3302.
## # i 3 more variables: deviance <dbl>, df.residual <int>, nobs <int>
```

The fitted model is

$$\widehat{medv} = \beta_0 + \beta_1 \cdot lstat$$
$$\widehat{medv} = 34.55 - 0.95 \cdot lstat$$

—

(b) Train / test split and evaluation

```r
# Split the data
boston_split <- initial_split(boston_df, prop = 0.80)
boston_train <- training(boston_split)
boston_test  <- testing(boston_split)

# Fit on training data
boston_fit_train <- lm_spec |>
  fit(medv ~ lstat, data = boston_train)

# Predictions using augment (same approach as class)
train_aug <- augment(boston_fit_train, new_data = boston_train)
test_aug  <- augment(boston_fit_train, new_data = boston_test)

# Performance metrics
train_aug |>
  metrics(truth = medv, estimate = .pred)
```

```
## # A tibble: 3 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rmse    standard       6.03
## 2 rsq     standard       0.555
## 3 mae     standard       4.34
```

```r
test_aug  |>
  metrics(truth = medv, estimate = .pred)
```

```
## # A tibble: 3 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rmse    standard       6.83
## 2 rsq     standard       0.510
## 3 mae     standard       5.05
```

Interpretation

- The training metrics show that `lstat` explains a substantial amount of variation in `medv`.
- Test performance is slightly worse than training performance, which is expected and indicates reasonable generalization.

(c) Confidence intervals for coefficients

```r
# Model fit on full data
tidy(boston_fit, conf.int = TRUE)
```

```
## # A tibble: 2 x 7
##   term        estimate std.error statistic   p.value conf.low conf.high
##   <chr>          <dbl>     <dbl>     <dbl>     <dbl>    <dbl>     <dbl>
## 1 (Intercept)   34.6      0.563       61.4 3.74e-236    33.4     35.7
## 2 lstat         -0.950    0.0387     -24.5 5.08e- 88    -1.03    -0.874
```

```r
# Model fit on training data
tidy(boston_fit_train, conf.int = TRUE)
```

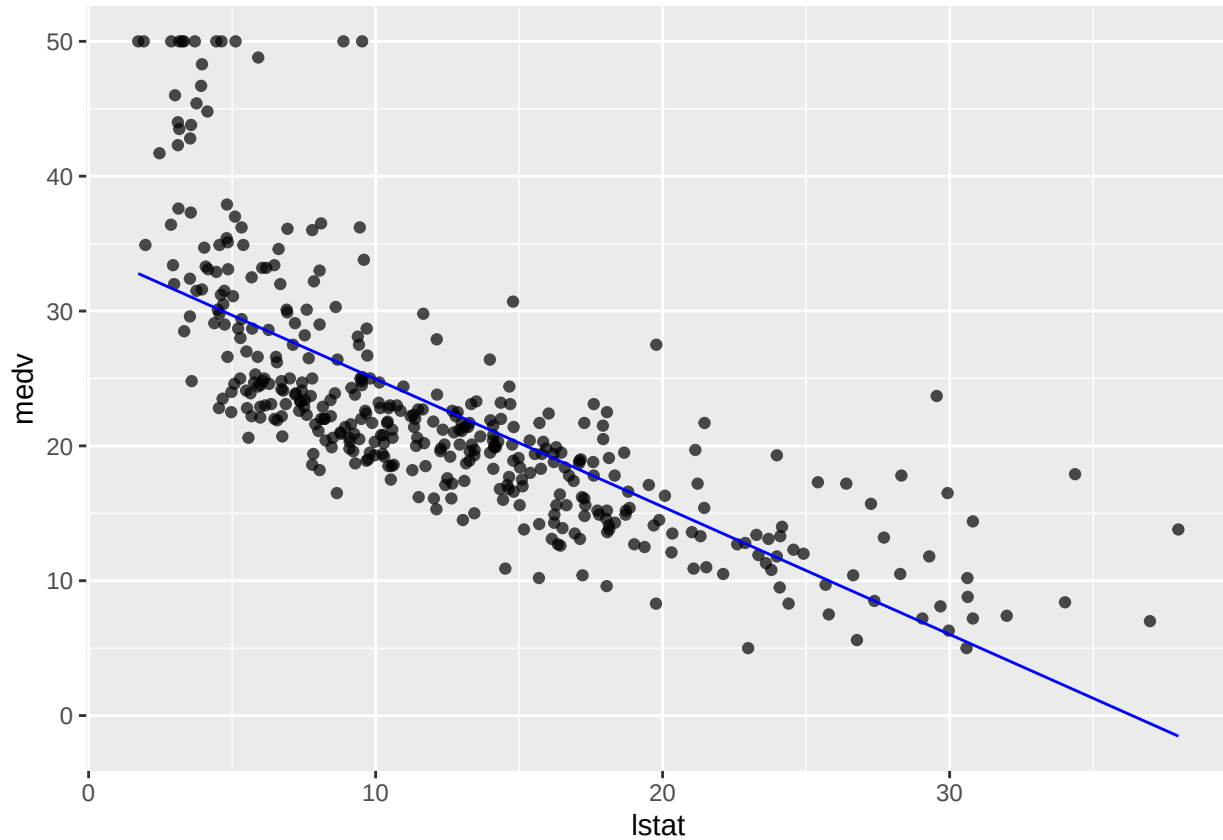
```
## # A tibble: 2 x 7
##   term        estimate std.error statistic   p.value conf.low conf.high
##   <chr>          <dbl>     <dbl>     <dbl>     <dbl>    <dbl>     <dbl>
## 1 (Intercept)   34.4      0.609       56.5 2.72e-193    33.2     35.6
## 2 lstat         -0.947    0.0423     -22.4 1.23e- 72    -1.03    -0.864
```

Interpretation

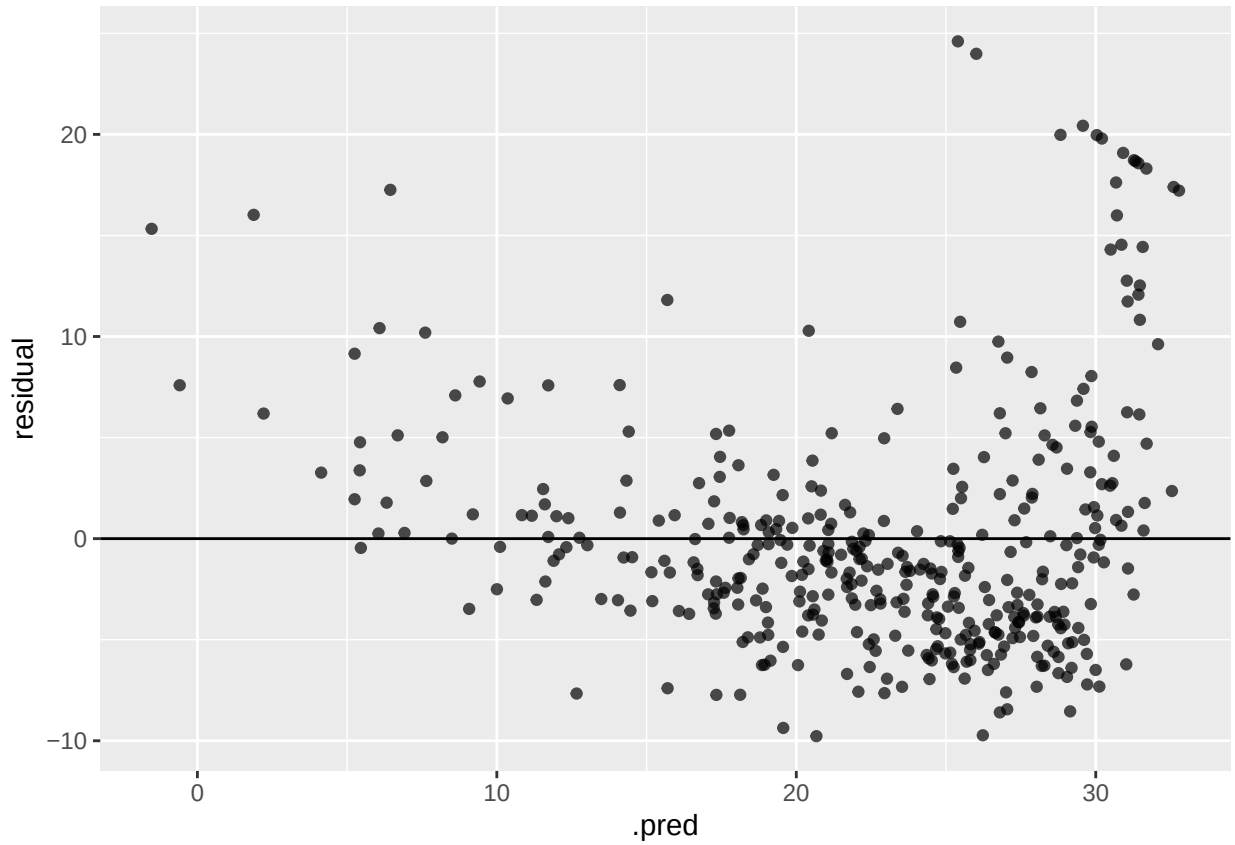
- The slope confidence interval does not include 0, indicating a clear negative association between `lstat` and `medv`.
- Practically, higher values of `lstat` are associated with lower median home values.

(d) Visualization and diagnostics

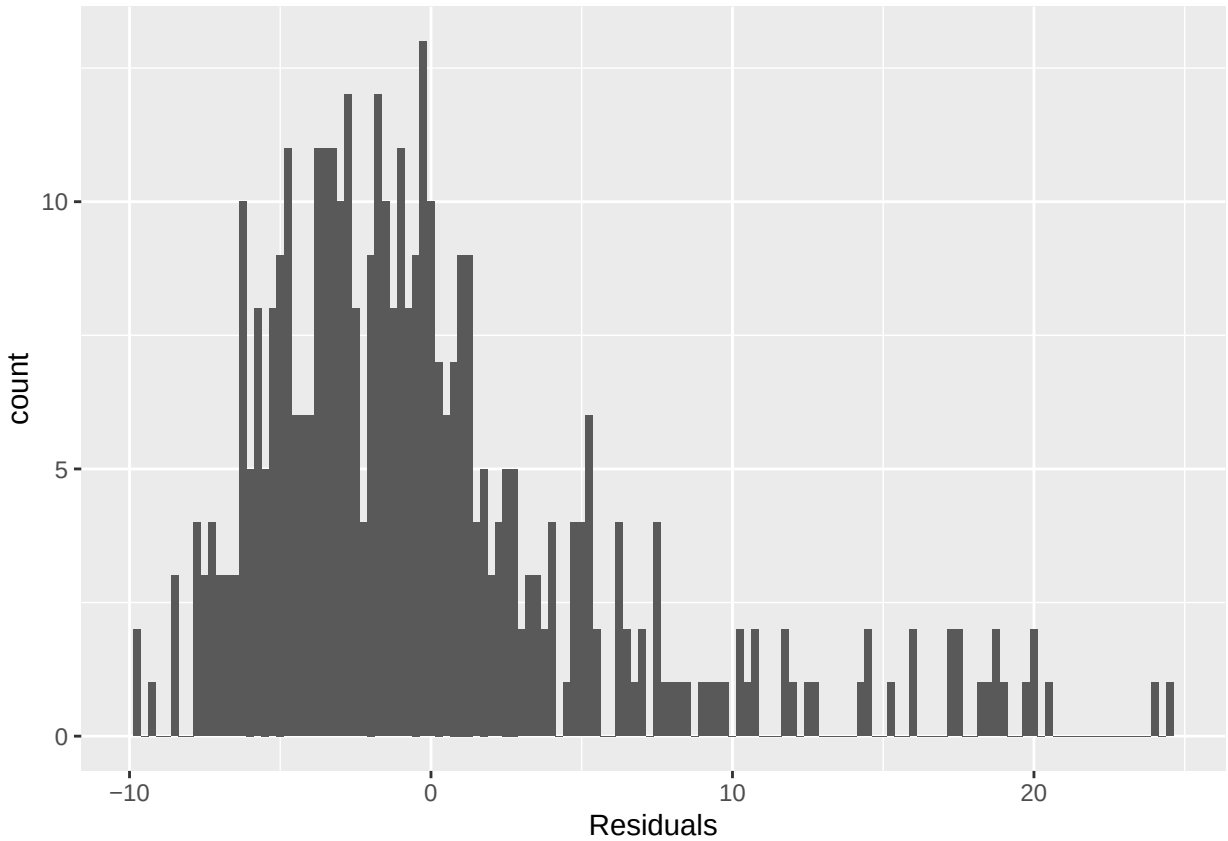
```r
# Scatterplot with fitted line
ggplot(train_aug, aes(x = lstat, y = medv)) +
  geom_point(alpha = 0.7) +
  geom_line(aes(y = .pred), color = "blue") +
  labs(x = "lstat", y = "medv")
```



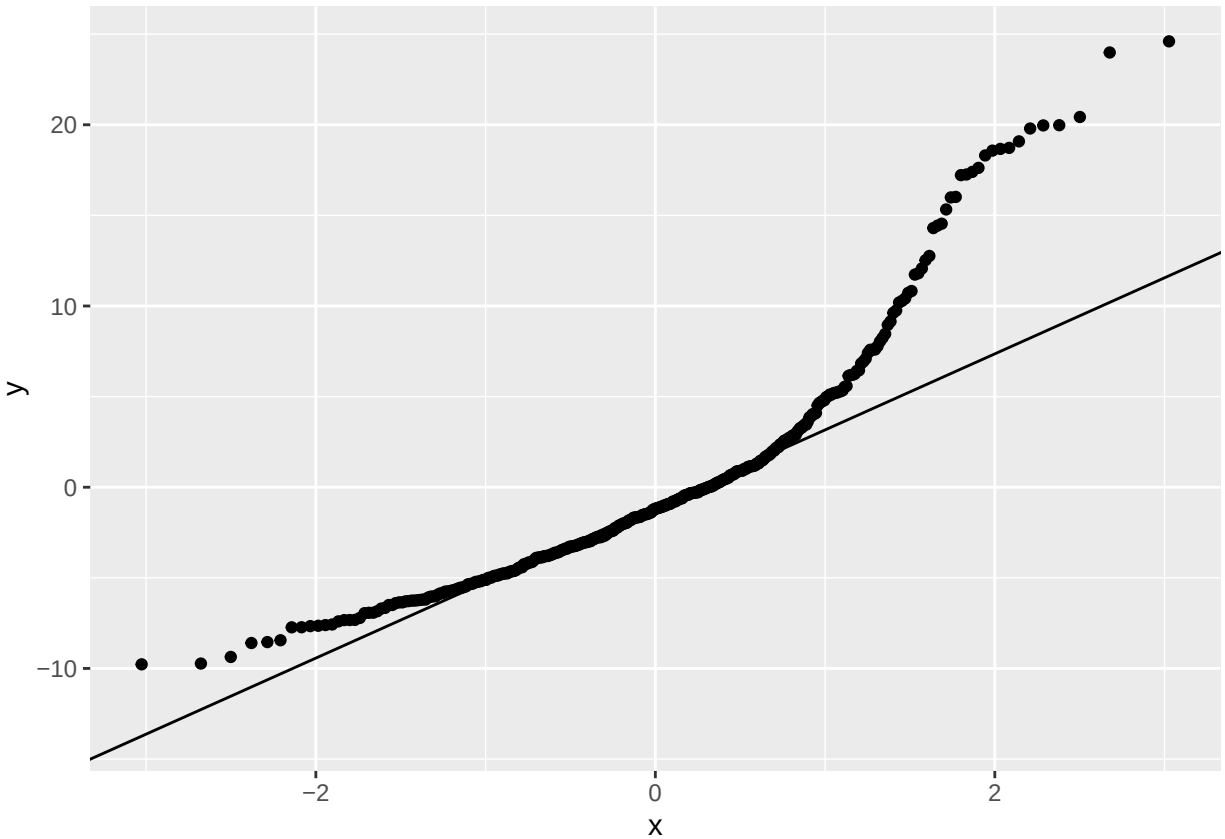

```r
# Residual analysis
train_aug <- train_aug |>
  mutate(residual = medv - .pred)

ggplot(train_aug, aes(x = .pred, y = residual)) +
  geom_point(alpha = 0.7) +
  geom_hline(yintercept = 0)
```




```
ggplot(train_aug, aes(x = residual)) +
  geom_histogram(binwidth = 0.25) +
  xlab("Residuals")
```




```
ggplot(train_aug, aes(sample = residual)) +
  stat_qq() +
  stat_qq_line()
```



Discussion

- `lstat` is a reasonable single predictor for `medv` and captures a strong overall trend.
- Residual plots suggest some curvature and changing variance, so a simple linear model does not fully capture the structure in the data.
- This model works well as a first pass, but a more flexible model could likely improve the fit.